

SAFE-RAG

Hallucination Detection & Fact Verification API for LLM Outputs

Type: Full-stack REST API + web frontend

Backend: Python / FastAPI • **Frontend:** Next.js / React / TypeScript

Classification engine: Gemini LLM (with deterministic rule-based fallback)

Reference basis: LeNguyenAnhKhoa/Hallucination-Detection (SciHal2025, A.M.P system)

Report date: September 2026

Contents

1. What This Project Does
2. System Architecture
3. How a Request Actually Flows Through the System
4. Backend — File by File
5. Frontend — File by File
6. The Swagger Docs Page (`/docs`) — What It Is and How to Use It Live
7. How to Run the Project
8. Suggested Presentation / Demo Script
9. Design Decisions & Known Limitations
10. Relationship to the Reference Paper
11. Anticipated Questions & Answers

1. What This Project Does

SAFE-RAG takes two pieces of text — a **source context** (e.g. a document, a database record, retrieved passages) and an **LLM-generated answer** that claims to be based on that context — and checks whether the answer is actually supported by the context.

It does this the way a fact-checker would: it splits the answer into individual factual claims (roughly one per sentence), and for each claim decides one of three verdicts:

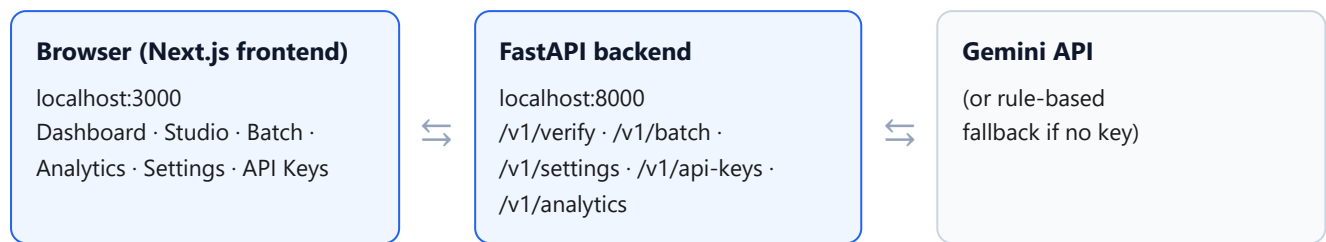
Verdict	Meaning
Supported	The context confirms this claim.
Contradicted	The context directly disagrees with this claim (a hallucination).
Abstain	The context has no information to confirm or deny this claim (e.g. a forecast, an opinion).

Each claim also gets:

- A **risk score** (0–100): how likely the claim is a hallucination.
- An **error type**: `numerical_mismatch` , `temporal_mismatch` , `entity_mismatch` , `formatting_mismatch` , `unverifiable` , or `none` .
- **Evidence**: a quote/paraphrase from the context justifying the score.

The project has two halves: a real **backend REST API** that does this classification work, and a **frontend dashboard** (already scaffolded before this work began) that was previously wired to fake `setTimeout` mock data. All six frontend pages now call the real backend.

2. System Architecture



The two halves are **separate processes** talking over plain HTTP + JSON, secured with a Bearer API key and CORS-restricted to the frontend's origin. This mirrors how a real product would be built: a thin client and an independently deployable, independently testable API service.

Persistence

There is no database. Three JSON files under **backend/data/** act as tiny collections:

File	Holds
<code>api_keys.json</code>	Issued API keys (id, secret, label, revoked flag, last-used time)
<code>settings.json</code>	The current risk-threshold configuration (lower/upper bound)
<code>history.json</code>	An append-only log of every verification run — this is what powers Analytics/Dashboard with <i>real</i> numbers

This is a deliberate simplification for a course project: fully inspectable, zero setup, easy to explain — not meant to be how a production system would store this data (see Section 9).

3. How a Request Actually Flows Through the System

Walking through one concrete example end to end — this is the best thing to narrate live in the demo.

Input

```
context: "Q3 revenue was $45M, reported on March 3, 2024 by CEO Jane Smith."  
answer: "The company revenue grew to $50M in Q3. The new product launched  
        in 2025. CEO John Doe announced record margins next year."
```

Step-by-step

1. **Claim decomposition** (`backend/app/services/claims.py`) splits the answer on sentence boundaries into 3 claims.
2. **Per-claim classification** (`backend/app/services/verifier.py`) sends each claim + the context to Gemini (`gemini_client.py`) asking for a strict JSON verdict: `{risk_score, error_type, evidence}`. If Gemini is unavailable or errors, the deterministic rule-based scorer (`rules.py`) takes over automatically.
3. **Threshold lookup**: the current `lower_bound` / `upper_bound` settings are read from `settings.json`. A risk score below the lower bound → *Supported*; above the upper bound → *Contradicted*; in between → *Abstain*.
4. **Result assembly**: claims are packaged into the API response and appended to `history.json` for analytics.

Actual output (captured from a real run of this backend)

```

{
  "claims": [
    {
      "text": "The company revenue grew to $50M in Q3.",
      "verdict": "Contradicted",
      "risk_score": 88.0,
      "error_type": "numerical_mismatch",
      "evidence": "Context contains different figures: 3, 45."
    },
    {
      "text": "The new product launched in 2025.",
      "verdict": "Contradicted",
      "risk_score": 82.0,
      "error_type": "temporal_mismatch",
      "evidence": "Context references different dates: 2024."
    },
    {
      "text": "CEO John Doe announced record margins next year.",
      "verdict": "Abstain",
      "risk_score": 55.0,
      "error_type": "unverifiable",
      "evidence": null
    }
  ],
  "hallucination_rate": 100.0,
  "engine": "rule_based_fallback"
}

```

Good demo moment: point out that the numeric mismatch (\$50M vs \$45M) and the date mismatch (2025 vs 2024) were correctly separated into different **error_type** categories, and the unverifiable forecast about "next year" correctly abstained instead of being falsely marked right or wrong.

4. Backend — File by File

All under `backend/app/`.

Entry point

File	Role
<code>main.py</code>	Creates the FastAPI app, adds CORS middleware, registers all routers, runs <code>ensure_default_key()</code> on startup so a working API key always exists.
<code>config.py</code>	Loads <code>.env</code> : Gemini key/model, allowed CORS origins, whether auth is required, history size limit.
<code>schemas.py</code>	Every Pydantic request/response model (<code>VerifyRequest</code> , <code>Claim</code> , <code>BatchResponse</code> , <code>ThresholdSettings</code> , <code>ApiKeyPublic</code> , analytics models, etc.) — this is the API's data contract.
<code>store.py</code>	Tiny generic JSON-file read/write helper with a thread lock, used by every service below.
<code>security.py</code>	API key lifecycle: create, list, roll (rotate secret), revoke, and the <code>require_api_key</code> dependency that every protected route depends on.

Services (`app/services/`) — the actual verification logic

File	Role
<code>claims.py</code>	Splits an answer string into individual claim sentences.
<code>gemini_client.py</code>	Builds the classification prompt and calls the Gemini API, asking for strict JSON. Returns <code>None</code> on any failure so the caller can fall back.
<code>rules.py</code>	Deterministic backup classifier: compares numbers, 4-digit years, and capitalized entities between claim and context using regex. No LLM call, no network — always available.
<code>verifier.py</code>	Orchestrator: decomposes the answer, tries Gemini per claim, falls back to rules, converts each risk score into a verdict using the current thresholds. This is the file that ties everything together.
<code>settings_service.py</code>	Get/save the threshold configuration.
<code>history.py</code>	Appends every verification run to the history log and reads it back for analytics.

Routers (`app/routers/`) — the HTTP surface

File	Endpoints
<code>verify.py</code>	<code>POST /v1/verify</code> — single context/answer pair

batch.py	POST /v1/batch — a list of pairs, returns per-row + aggregate summary
settings_router.py	GET / PUT /v1/settings
api_keys.py	GET/POST /v1/api-keys , /roll , /revoke , /bootstrap
analytics.py	GET /v1/analytics/summary , /timeseries , /models — all computed live from history.json

5. Frontend — File by File

Next.js App Router, all pages are client components (`"use client"`) since they need browser `fetch` /state.

Shared code

File	Role
<code>lib/api.ts</code>	The one place that talks to the backend. Wraps <code>axios</code> , attaches the API key to every request, auto-provisions a key on first load (<code>ensureApiKey</code>), and self-heals : if a request 401s, it silently mints a fresh key and retries once. Exports typed functions for every endpoint (<code>verify</code> , <code>batchVerify</code> , <code>getSettings</code> , <code>listApiKeys</code> , <code>getAnalyticsSummary</code> , etc.).
<code>lib/csv.ts</code>	A small hand-written CSV parser (handles quoted fields) plus a CSV serializer, used by the Batch page to read uploads and export results.
<code>components/Sidebar.tsx</code>	Left navigation, unchanged from the original scaffold.

Pages (`app/`)

Route	File	What it does now (vs. before)
<code>/</code>	<code>app/page.tsx</code>	Dashboard. Fetches <code>/v1/analytics/summary</code> on load: real claim counts, real hallucination rate, a real error-type pie chart, and a real recent-activity feed. Was: hardcoded numbers.
<code>/studio</code>	<code>app/studio/page.tsx</code>	Verification Studio. Textareas for context/answer + a model-name field, calls <code>verify()</code> , renders the real claims with verdict color, risk gauge, evidence, and error type. Was: a 2-second fake timer returning 3 hardcoded claims.
<code>/batch</code>	<code>app/batch/page.tsx</code>	Drag-and-drop CSV/JSON upload, parsed client-side, sent to <code>/v1/batch</code> , renders a real results table and exports a real CSV report. Was: a fake progress bar with 2 hardcoded rows.
<code>/analytics</code>	<code>app/analytics/page.tsx</code>	Fetches time series, error breakdown, and per-model stats — all from real logged runs. Was: 3 hardcoded chart datasets.
<code>/settings</code>	<code>app/settings/page.tsx</code>	Loads/saves the real risk thresholds via <code>GET / PUT /v1/settings</code> — moving the sliders changes live classification behavior. Was: local-only React state, never sent anywhere.
<code>/api-keys</code>	<code>app/api-keys/page.tsx</code>	Real key list/create/roll/revoke against the backend, plus a live curl example pointed at the actual API base URL. Was: one hardcoded fake key string.

6. The Swagger Docs Page (`/docs`)

FastAPI auto-generates this page from the Pydantic schemas and route definitions in the code — nothing was hand-written for it. Open **`http://127.0.0.1:8000/docs`** while the backend is running.

What's on it

- Every endpoint, grouped by tag (`verify` , `batch` , `settings` , `api-keys` , `analytics`).
- The exact request body schema and response schema for each — generated straight from `schemas.py` , so it can never drift out of sync with the real API.
- A "Try it out" button on every endpoint that sends a real request to your running backend and shows the real response, right in the browser — no Postman or curl needed.

What to do with it during the presentation

1. Expand `POST /v1/verify` , click **Try it out**.
2. Paste a context/answer pair into the example body and click **Execute**.
3. Scroll down and show the real JSON response with verdicts, risk scores, and error types.

Why this matters for an API course: it proves the system is a genuine, independently usable REST API — not logic baked into the frontend. Anyone (a mobile app, another service, a grading script) could call this API directly, with no browser UI involved at all.

One thing to know: `/v1/verify` requires an `Authorization: Bearer <key>` header. Grab a working key first from `POST /v1/api-keys/bootstrap` (no auth needed) or from the Developer API page in the frontend, then paste it into the padlock/Authorize button at the top of the Swagger page before calling protected endpoints.

7. How to Run the Project

Terminal 1 — backend

```
cd backend
.venv\Scripts\activate
uvicorn app.main:app --reload --port 8000
```

Ready when you see `Application startup complete.` Verify: <http://127.0.0.1:8000/v1/health> should return `{"status": "ok"}`.

Terminal 2 — frontend

```
npm run dev
```

Open <http://localhost:3000>.

Works with **zero configuration** — no Gemini key needed. Without one, every claim is scored by the rule-based fallback automatically. To use real Gemini classification, get a free key at aistudio.google.com/apikey and paste it into `backend/.env` as `GEMINI_API_KEY`, then restart the backend terminal.

8. Suggested Presentation / Demo Script

1. **Open with the problem:** LLMs hallucinate; RAG systems need a verification layer that checks generated answers against retrieved context before showing them to a user.
2. **Show the Dashboard** (empty state is fine) — explain it will fill in live as you run verifications.
3. **Verification Studio:** paste the example from Section 3, click Run, narrate the three verdicts and why each error type was assigned.
4. **Settings:** drag the risk threshold sliders, re-run the same claim, show the verdict actually changes — proves the thresholds are live, not decorative.
5. **Batch Processing:** upload `public/sample-batch.csv`, show the results table and CSV export.
6. **Analytics & Dashboard:** refresh — now populated with the real runs you just did.
7. **Developer API page & /docs** : show the API key, then hit `/v1/verify` directly from Swagger — proves this is a real, independently callable API, not UI-only logic.
8. **Close** with the architecture diagram (Section 2) and the design decisions (Section 9) to show you understand the trade-offs you made.

9. Design Decisions & Known Limitations

Decision	Why	What a production system would do instead
Sentence-level claim splitting (regex)	Simple, fast, good enough for short LLM answers	LLM-based atomic sub-claim decomposition
JSON files instead of a database	Zero setup, fully inspectable for a demo	Postgres/SQLite with proper concurrency
Rule-based fallback (regex number/date/entity overlap)	Keeps the demo working with no API key, no internet, or a rate-limited key	A real trained NLI/entailment model as the fallback, not just the safety net
Plaintext API keys in a JSON file	Easy to reason about and demo	Hashed keys, a real auth/session system for the key-management endpoints
One Gemini call per claim	Simple, clear per-claim evidence	Batched/parallel calls for large-scale batch jobs

10. Relationship to the Reference Paper

The reference repository ([LeNguyenAnhKhoa/Hallucination-Detection](#)) is the code for "**A.M.P at SciHal2025: Automated Hallucination Detection in Scientific Content via LLMs and Prompt Engineering**" (ACL 2025 SDP Workshop) — 4th place in Subtask 1 and 2nd place in Subtask 2 of the SciHal2025 shared task.

It is **not** a REST API — it's a set of research scripts that call the Gemini/OpenAI APIs with few-shot prompts to classify a claim vs. a reference as **Entailment**, **Contradiction**, or **Unverifiable**.

Reference paper concept	This project's equivalent
Entailment	Supported
Contradiction	Contradicted
Unverifiable	Abstain
Few-shot prompt asking for a label	<code>gemini_client.py</code> 's structured JSON prompt asking for a risk score + error type

What this project adds on top of that idea: a full serving layer (FastAPI + routing + auth), a threshold-based verdict system instead of the LLM deciding the label directly, a rule-based fallback for resilience, persistence, and a connected frontend — none of which exist in the reference repo.

11. Anticipated Questions & Answers

Q: Why does the LLM output a risk score instead of the verdict directly?

A: So the Supported/Contradicted/Abstain cutoffs are configurable at runtime (via Settings) without needing to change the prompt or re-call the model — the same score can be re-classified under different thresholds instantly.

Q: What happens with no internet / no API key?

A: `gemini_client.py` returns `None` on any failure, and `verifier.py` falls back to `rules.py` automatically per-claim — the API never errors out because of the LLM being unavailable.

Q: How is this secured?

A: Bearer API-key auth on every verification/analytics/settings endpoint, CORS restricted to the frontend's origin only, keys can be rotated or revoked. Key-management endpoints themselves are intentionally open (like a dashboard using session auth) — documented as a simplification in Section 9.

Q: Does it scale?

A: Not as-is — JSON-file storage assumes a single process. The architecture (stateless FastAPI routes, a swappable classification service) would scale by swapping the JSON store for a real database; the API contract wouldn't need to change.

Q: How accurate is the rule-based fallback vs. the LLM?

A: The fallback is intentionally crude (regex overlap) — a safety net for demo resilience, not a competitor to the LLM path. The `engine` field in every response reports which one was actually used, so this is fully transparent.